Threads: Synchronisation: Verteilter Zugriff auf gemeinsame Ressourcen

Carsten Gips (HSBI)

TL;DR

Bei verteiltem Zugriff auf gemeinsame Ressourcen besteht Synchronisierungsbedarf, insbesondere sollten nicht mehrere Threads gleichzeitig geteilte Daten modifizieren. Dazu kommt das Problem, dass ein Thread in einer komplexen Folge von Aktionen die Zeitscheibe verlieren kann und dann später mit veralteten Daten weiter macht.
Um den Zugriff auf gemeinsame Ressourcen oder den Eintritt in kritische Bereiche zu schützen und zu synchronisieren, kann man diese Zugriffe oder Bereiche in einen `synchronized`-Block legen. Dazu benötigt man noch ein beliebiges (gemeinsam sichtbares) Objekt, welches als Wächter- oder Sperr-Objekt fungiert. Beim Eintritt in den geschützten Block muss ein Thread einen Lock auf dem Sperr-Objekt erlangen. Hat bereits ein anderer Thread den Lock, wird der neue Thread so lange blockiert, bis der Lock wieder "frei" ist. Beim Eintritt in den Bereich wird dann durch den Thread auf dem Sperr-Objekt der Lock gesetzt und beim Austritt automatisch wieder aufgehoben. Dies nennt man auch **mehrseitige Synchronisierung** (mehrere Threads "stimmen" sich quasi untereinander über den Zugriff auf eine Ressource ab).
Um auf den Eintritt eines Ereignisses oder die Erfüllung einer Bedingung zu warten, kann man `wait` und `notify` nutzen. In einem `synchronized`-Block prüft man, ob die Bedingung erfüllt oder ein Ereignis eingetreten ist, und falls ja arbeitet man damit normal weiter. Falls die Bedingung nicht erfüllt ist oder das Ereignis nicht eingetreten ist, kann man auf dem im `synchronized`-Block genutzten Sperr-Objekt die Methode `wait()` aufrufen. Damit wird der Thread in die entsprechende Schlange auf dem Sperr-Objekt eingereiht und blockiert. Zusätzlich wird der Lock auf dem Sperr-Objekt freigegeben. Zum "Aufwecken" nutzt man an geeigneter Stelle auf dem **selben Sperr-Objekt** die Methode `notify()` oder `notifyALl()` (erstere weckt einen in der Liste des Sperr-Objekts wartenden Thread, die letztere alle). Nach dem Aufwachen macht der Thread nach seinem `wait()` weiter. Es ist also wichtig, dass die Bedingung, wegen der ursprünglich das `wait()` aufgerufen wurde, erneut abgefragt wird und ggf. erneut in das `wait()` gegangen wird. Dies nennt man **einseitige Synchronisierung**.
Es gibt darüber hinaus viele weitere Mechanismen und Probleme, die aber den Rahmen dieser Lehrveranstaltung deutlich übersteigen. Diese werden teilweise in den Veranstaltungen "Betriebssysteme" und/oder "Verteilte Systeme" besprochen.

Videos

- VL Threads Synchronisation
- Demo Teaser: Falscher Zugriff auf gemeinsame Ressourcen
- Demo Mehrseitige Synchronisation (Sperr-Objekt, synchronisierte Methode)
- Demo Mehrseitige Synchronisation: Deadlock
- Demo Einseitige Synchronisation

Unterlagen

- PDF-Version

---

1 Motivation: Verteilter Zugriff auf gemeinsame Ressourcen

```
public class Teaser implements Runnable {
    private int val = 0;

    public static void main(String... args) {
        Teaser x = new Teaser();
        new Thread(x).start();
```

```
7           new Thread(x).start();
8       }
9
10      private void incrVal() {
11          ++val;
12          System.out.println(Thread.currentThread().getId() + ": " + val);
13      }
14
15      public void run() {
16          IntStream.range(0, 5).forEach(i -> incrVal());
17      }
18  }
```

Demo: synchronised.Teaser

2 Zugriff auf gemeinsame Ressourcen: Mehrseitige Synchronisierung

```
1  synchronized (<Object reference>) {
2      <statements (synchronized)>
3  }
```

=> **“Mehrseitige Synchronisierung”**

Fallunterscheidung: Thread T1 führt `synchronized`-Anweisung aus:

- Sperre im Sperr-Objekt nicht gesetzt:
  1. T1 setzt Sperre beim Eintritt,
  2. führt den Block aus, und
  3. löst Sperre beim Verlassen
- Sperre durch T1 gesetzt:
  1. T1 führt den Block aus, und
  2. löst Sperre beim Verlassen **nicht**
- Sperre durch T2 gesetzt: => T1 wird blockiert, bis T2 die Sperre löst

Anmerkung: Das für die Synchronisierung genutzte Objekt nennt man “Wächter-Objekt” oder auch “Sperr-Objekt” oder auch “Synchronisations-Objekt”.

Damit könnte man den relevanten Teil der Methode `incrVal()` beispielsweise in einen geschützten Bereich einschließen und als Sperr-Objekt das eigene Objekt (`this`) einsetzen:

```
1      private void incrVal() {
2          synchronized (this) { ++val; }
3      }
```

Demo: synchronised.ObjSync

3 Synchronisierte Methoden

```
void f() {
    synchronized (this) {
        ...
    }
}
```

… ist äquivalent zu …

```
synchronized void f() {
    ...
}
```

Kurzschreibweise: Man spart das separate Wächter-Objekt und synchronisiert auf sich selbst …

Die Methode `incrVal()` könnte entsprechend so umgeschrieben werden:

```
    private synchronized void incrVal() {
        ++val;
    }
```

Demo: synchronised.MethodSync

4 Probleme bei der (mehrseitigen) Synchronisierung: Deadlocks

```
public class Deadlock {
    private final String name;

    public synchronized String getName() { return name; }
    public synchronized void foo(Deadlock other) {
        System.out.format("%s: %s.foo() \n", Thread.currentThread().getName(),
            name);
        System.out.format("%s: %s.name()\n", Thread.currentThread().getName(),
            other.getName());
    }

    public static void main(String... args) {
        final Deadlock a = new Deadlock("a");
        final Deadlock b = new Deadlock("b");

        new Thread(() -> a.foo(b)).start();
        new Thread(() -> b.foo(a)).start();
    }
}
```

Viel hilft hier nicht viel! Durch zu großzügige mehrseitige Synchronisierung kann es passieren, dass Threads gegenseitig aufeinander warten: Thread A belegt eine Ressource, die ein anderer Thread B haben möchte und Thread B belegt eine Ressource, die A gerne bekommen würde. Da es dann nicht weitergeht, nennt man diese Situation auch “Deadlock” (“Verklemmung”).

Im Beispiel ruft der erste Thread für das Objekt a die `foo()`-Methode auf und holt sich damit den Lock auf a. Um die Methode beenden zu können, muss noch die `getName()`-Methode vom Objekt b durch diesen ersten Thread aufgerufen werden. Dafür muss der erste Thread den Lock auf b bekommen.

Dummerweise hat parallel der zweite Thread auf dem Objekt b die `foo()`-Methode aufgerufen und sich damit den Lock auf b geholt. Damit muss der erste Thread so lange warten, bis der zweite Thread den Lock auf b freigibt.

Das wird allerdings nicht passieren, da der zweite Thread zur Beendigung der `foo()`-Methode noch `getName()` auf a ausführen muss und dazu den Lock auf b holen, den aber aktuell der erste Thread hält.

Und schon geht's nicht mehr weiter :-)

Demo: synchronised.Deadlock

5 Warten auf andere Threads: Einseitige Synchronisierung

5.1 Problem

- Thread T1 wartet auf Arbeitsergebnis von T2
- T2 ist noch nicht fertig

5.2 Mögliche Lösungen

1. Aktives Warten (Polling): Permanente Abfrage
   - Kostet unnötig Rechenzeit
2. Schlafen mit `Thread.sleep()`
   - Etwas besser; aber wie lange soll man idealerweise schlafen?
3. Warten mit `T2.join()`
   - Macht nur Sinn, wenn T1 auf das *Ende* von T2 wartet
4. **Einseitige Synchronisierung** mit `wait()` und `notify()`
   - Das ist DIE Lösung für das Problem :)

6 Einseitige Synchronisierung mit *wait* und *notify*

- **wait**: Warten auf Erfüllung einer Bedingung (Thread blockiert):

```java
synchronized (obj) {    // Geschützten Bereich betreten
    while (!condition) {
        try {
            obj.wait(); // Thread wird blockiert
        } catch (InterruptedException e) {}
    }
    ...     // Condition erfüllt: Tue Deine Arbeit
}
```

=> Bedingung nach Rückkehr von `wait` erneut prüfen!

6.1 Eigenschaften von *wait*

- Thread ruft auf Synchronisations-Objekt die Methode `wait` auf
- Prozessor wird entzogen, Thread blockiert
- Thread wird in interne Warteschlange des Synchronisations-Objekts eingetragen
- Sperre auf Synchronisations-Objekt wird freigegeben

=> Geht nur innerhalb der **`synchronized`**-Anweisung für das Synchronisations-Objekt!

7 Einseitige Synchronisierung mit *wait* und *notify* (cnt.)

- **notify**: Aufwecken von wartenden (blockierten) Threads:

```java
synchronized (obj) {
    obj.notify();       // einen Thread "in" obj aufwecken
    obj.notifyAll();    // alle Threads "in" obj wecken
}
```

7.1 Eigenschaften von *notify* bzw. *notifyAll*

- Thread ruft auf einem Synchronisations-Objekt die Methode `notify` oder `notifyAll` auf
- Falls Thread(s) in Warteschlange des Objekts vorhanden, dann
  - `notify`: Ein *zufälliger* Thread wird aus Warteschlange entfernt und in den Zustand “ausführungsbereit” versetzt
  - `notifyAll`: Alle Threads werden aus Warteschlange entfernt und in den Zustand “ausführungsbereit” versetzt

=> Geht nur innerhalb der **`synchronized`**-Anweisung für das Synchronisations-Objekt!

Demo: synchronised.Staffel

8 Wrap-Up

Synchronisierungsbedarf bei verteiltem Zugriff auf gemeinsame Ressourcen:

- Vorsicht mit konkurrierendem Ressourcenzugriff:
  Synchronisieren mit `synchronized` => **Mehrseitige Synchronisierung**
- Warten auf Ereignisse mit `wait` und `notify`/`notifyAll` => **Einseitige Synchronisierung**

9 Zum Nachlesen

- Oracle Corporation (2024)
- Boles (2008)

Lernziele

- k2: Ich kann die Notwendigkeit zur Synchronisation erklären
- k2: Ich kann den Unterschied zwischen einseitiger und mehrseitiger Synchronisation erklären
- k3: Ich kann die Synchronisation mit synchronized, wait, notify und notifyAll praktisch einsetzen

Challenges

Hamster-Welt

In den Vorgaben finden Sie eine Modellierung für eine Hamsterwelt.
Es gibt rote und blaue Hamster, die sich unabhängig von einander bewegen können. Es gibt einen Tunnel, den die Hamster betreten und durchqueren können. In der Vorgabe ist ein kleines Hauptprogramm enthalten, welches einige Hamster anlegt und herumlaufen lässt.

Teil I: Stau im Tunnel

Die Hamster sind sehr neugierig und wollen gern durch den Tunnel gehen, um die Höhle auf der anderen Seite zu erkunden. Leider mussten sie feststellen, dass immer nur ein Hamster zu einem Zeitpunkt im Tunnel sein darf, sonst wird die Luft zu knapp.
Ergänzen Sie die Vorgaben, so dass sich immer nur ein paralleler Hamster (egal welcher Farbe) im Tunnel aufhalten kann. Wenn ein Hamster in den Tunnel will, aber nicht hinein kann, dann soll er am Eingang warten, also nicht noch einmal in seiner Höhle herumlaufen. (Das passiert eigentlich automatisch, wenn Sie alles richtig machen.)

Teil II: Schlaue Hamster

Die Hamster sind schlau und haben bemerkt, dass die Einschränkung aus der letzten Aufgabe zu stark war. Sie überleben auch, wenn sich beliebig viele blaue Hamster oder nur genau ein roter Hamster im Tunnel aufhalten.
Erweitern Sie die Implementierung aus der letzten Aufgabe, so dass folgende Bedingungen eingehalten werden:

- Es dürfen sich beliebig viele blaue Hamster gleichzeitig im Tunnel befinden.
  Das bedeutet, dass in diesem Fall zwar weitere blaue Hamster den Tunnel betreten dürfen, aber kein roter Hamster in den Tunnel hinein darf.
- Wenn sich ein roter Hamster im Tunnel aufhält, dürfen keine anderen Hamster (unabhängig von deren Farbe) den Tunnel betreten.

Quellen

Boles, D. 2008. *Parallele Programmierung spielend gelernt mit dem Java-Hamster-Modell*. Vieweg+Teubne. http://www.boles.de/hamster/band3.html.

Oracle Corporation. 2024. „The Java Tutorials“. 2024. https://docs.oracle.com/javase/tutorial/.

Unless otherwise noted, this work is licensed under CC BY-SA 4.0.

> **Last modified:** 316c6e1 (lecture: improve learning outcomes (Threads Syncr), 2025-05-26)